DP Computer Science: A 4.3.3 Model Evaluation & Hyperparameter Tuning

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Explain why **accuracy** can be a misleading metric for evaluating classification models, especially with imbalanced data
- Describe **precision, recall, and F1 score** as key evaluation metrics for classification
- Analyse a **confusion matrix** to calculate accuracy, precision, and recall
- Describe the concepts of **overfitting** and **underfitting** in the context of model training
- Explain that **hyperparameter tuning** is the process of finding the optimal settings for a model to improve its performance

Key Vocabulary

Term	Definition
Confusion matrix	A table that summarises the performance of a classification model by comparing predicted vs. actual classes
True Positive (TP)	Correctly predicted positive cases
True Negative (TN)	Correctly predicted negative cases
False Positive (FP)	Incorrectly predicted positive cases (Type I error)
False Negative (FN)	Incorrectly predicted negative cases (Type II error)
Accuracy	(TP + TN) / Total predictions — the proportion of correct predictions
Precision	TP / (TP + FP) — the proportion of positive predictions that are correct
Recall (Sensitivity)	TP / (TP + FN) — the proportion of actual positives that are correctly identified
F1 Score	The harmonic mean of precision and recall; balances both metrics
Overfitting	Model learns noise in training data and performs poorly on new data
Underfitting	Model is too simple and fails to capture patterns in the data
Hyperparameter	A configurable setting of a model that is set before training (e.g., k in K-NN)
Hyperparameter tuning	The process of finding the optimal hyperparameter values to improve model performance

Approaches to Learning (ATL) Elements

Skill	Description
Thinking (Critical Thinking & Analysis)	Critically evaluating a model's performance beyond simple accuracy; analysing trade-offs between precision and recall
Problem-Solving	Applying formulas to a confusion matrix to diagnose a model's strengths and weaknesses

---

2. The Hook: Why Accuracy Can Be Misleading

The Medical Diagnosis Scenario

> *"A model is **99% accurate** at detecting a rare, dangerous disease. Is this a good model?"*

```text
┌─────────────────────────────────────────────────────────────────┐
│             THE ACCURACY PARADOX                                │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    Scenario: A rare disease affects only 1 in 1000 people     │
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │  A model that ALWAYS predicts "No Disease":         │  │
│    │                                                         │  │
│    │    • 1000 people tested                             │  │
│    │    • 999 are healthy (correctly predicted)          │  │
│    │    • 1 has the disease (incorrectly predicted)      │  │
│    │                                                         │  │
│    │    Accuracy = 999 / 1000 = 99.9%                    │  │
│    │    But it missed EVERY sick person!                 │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
│    ❌ Accuracy is high, but the model is USELESS!          │  │
│    ✅ We need better metrics to evaluate the model!        │  │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Key Insight

> *"Accuracy alone is not enough—especially when dealing with imbalanced datasets. We need metrics that tell us more about the model's performance."*

---

3. The Confusion Matrix

What is a Confusion Matrix?

> A confusion matrix is a table that summarises the performance of a classification model by comparing predicted classes against actual classes.

The Four Outcomes

```text
┌─────────────────────────────────────────────────────────────────┐
│                    CONFUSION MATRIX                             │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
```

```
|                              Predicted                            |
|                      Positive        Negative                     |
|              ┌──────────────────────────────────────────┐         |
|              │             │             │              │  |
|    Actual    │   True      │   False     │              │  |
|    Positive  │   Positive  │   Negative  │              │  |
|              │   (TP)      │   (FN)      │              │  |
|              ├─────────────┼─────────────┤              │  |
|              │             │             │              │  |
|    Actual    │   False     │   True      │              │  |
|    Negative  │   Positive  │   Negative  │              │  |
|              │   (FP)      │   (TN)      │              │  |
|              └─────────────┴─────────────┘              │  |
|                                                                   |
└───────────────────────────────────────────────────────────────────┘
```

Understanding Each Outcome

Outcome	Description	Example (Disease Test)
True Positive (TP)	Correctly predicted positive	Test says "Sick" and patient IS sick
True Negative (TN)	Correctly predicted negative	Test says "Healthy" and patient IS healthy
False Positive (FP)	Incorrectly predicted positive (Type I error)	Test says "Sick" but patient is healthy
False Negative (FN)	Incorrectly predicted negative (Type II error)	Test says "Healthy" but patient IS sick

Visual Example

```text
┌───────────────────────────────────────────────────────────────────┐
|             CONFUSION MATRIX EXAMPLE                              |
├───────────────────────────────────────────────────────────────────┤
|                                                                   |
|    Spam Filter Performance                                        |
|                                                                   |
|                              Predicted                            |
|                      Spam          Not Spam                       |
|              ┌──────────────────────────────────────────┐         |
|              │             │             │              │  |
|    Actual    │     TP      │     FN      │              │  |
|    Spam      │     90      │     10      │              │  |
|              ├─────────────┼─────────────┤              │  |
|              │             │             │              │  |
|    Actual    │     FP      │     TN      │              │  |
|    Not Spam  │     15      │    185      │              │  |
|              └─────────────┴─────────────┘              │  |
|                                                                   |
|    TP = 90 (correctly caught spam)                           |  |
|    FN = 10 (spam that slipped through)                       |  |
|    FP = 15 (legitimate emails marked as spam)               |  |
|    TN = 185 (correctly identified legitimate emails)        |  |
|                                                                   |
└───────────────────────────────────────────────────────────────────┘
```

4. Evaluation Metrics

Accuracy

Aspect	Description
Definition	The proportion of correct predictions
Formula	(TP + TN) / (TP + TN + FP + FN)
When to Use	When classes are balanced; not good for imbalanced data

```python
Accuracy = (TP + TN) / (TP + TN + FP + FN)

# Example from above:
Accuracy = (90 + 185) / (90 + 185 + 15 + 10)
        = 275 / 300
        = 91.7%
```

Precision

Aspect	Description
Definition	The proportion of positive predictions that are correct
Formula	TP / (TP + FP)
When to Use	When false positives are costly (e.g., marking legitimate email as spam)
Key Question	"When the model says 'yes', how often is it right?"

```python
Precision = TP / (TP + FP)

# Example from above:
Precision = 90 / (90 + 15)
         = 90 / 105
         = 85.7%
```

Recall (Sensitivity)

Aspect	Description
Definition	The proportion of actual positives that are correctly identified
Formula	TP / (TP + FN)
When to Use	When false negatives are costly (e.g., missing a disease)
Key Question	"How good is the model at finding ALL the 'yes' cases?"

```python
Recall = TP / (TP + FN)
```

```
# Example from above:
Recall = 90 / (90 + 10)
       = 90 / 100
       = 90.0%
```

---

5. Precision vs. Recall: The Trade-off

The Trade-off

```text
┌──────────────────────────────────────────────────────────────────┐
│              PRECISION vs. RECALL TRADE-OFF                      │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    ┌───────────────────────────────────────────────────────┐     │
│    │             HIGH PRECISION                            │     │
│    │                                                       │     │
│    │    • When the model says "yes", it's almost always    │     │
│    │      correct                                          │     │
│    │    • But it might miss some "yes" cases               │     │
│    │    • Minimises false positives                        │     │
│    └───────────────────────────────────────────────────────┘     │
│                                                                  │
│    ┌───────────────────────────────────────────────────────┐     │
│    │             HIGH RECALL                               │     │
│    │                                                       │     │
│    │    • The model finds almost all "yes" cases           │     │
│    │    • But it might incorrectly label some "no" cases   │     │
│    │      as "yes"                                         │     │
│    │    • Minimises false negatives                        │     │
│    └───────────────────────────────────────────────────────┘     │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

When to Prioritise Each

Scenario	Priority	Why?
Medical Diagnosis	Recall	Missing a disease (false negative) is dangerous; false positives can be investigated later
Spam Detection	Precision	Marking legitimate email as spam (false positive) is disruptive; missing some spam is less serious
Fraud Detection	Recall	Missing fraudulent transactions (false negatives) is costly
Search Engine	Precision	Irrelevant results (false positives) frustrate users

---

6. F1 Score

What is the F1 Score?

> The F1 Score is the harmonic mean of precision and recall. It provides a single metric that balances both precision and recall.

Formula

```python
F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

Why Use F1?

Aspect	Description
Balanced	Considers both precision and recall
Harmonic Mean	Penalises extreme values; a low precision or recall pulls down the F1
Single Score	Provides one number to compare models

Example

```python
# From the spam filter example:
Precision = 0.857 (85.7%)
Recall = 0.900 (90.0%)

F1 = 2 × (0.857 × 0.900) / (0.857 + 0.900)
   = 2 × 0.771 / 1.757
   = 1.542 / 1.757
   = 0.878 (87.8%)
```

---

7. Overfitting and Underfitting

What are Overfitting and Underfitting?

```text
┌─────────────────────────────────────────────────────────────────┐
│              OVERFITTING vs. UNDERFITTING                       │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │              UNDERFITTING                               │  │
│    │                                                         │  │
│    │   • Model is too simple                                 │  │
│    │   • Fails to capture patterns in the data               │  │
│    │   • Poor performance on BOTH training and test data     │  │
│    │   • "Too general"                                       │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │              OVERFITTING                                │  │
│    │                                                         │  │
│    │   • Model is too complex                                │  │
│    │   • Learns noise in the training data                   │  │
│    │   • Excellent on training data, poor on test data       │  │
│    │   • "Too specific"                                      │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
```

```
│     │           GOOD FIT                         │  │
│     │                                             │  │
│     │    • Model captures true patterns          │  │
│     │    • Good performance on BOTH training and test data  │  │
│     │    • "Just right"                          │  │
│     └─────────────────────────────────────────────┘  │
│                                                      │
└──────────────────────────────────────────────────────┘
```

Visual Examples

```text

┌──────────────────────────────────────────────────────┐
│              VISUAL EXAMPLES                          │
├──────────────────────────────────────────────────────┤
│                                                      │
│     ┌─────────────────────────────────────────────┐  │
│     │           UNDERFITTING                      │  │
│     │                                             │  │
│     │    •    •    •    •    •                   │  │
│     │       • • • • • •                           │  │
│     │    ────•────•────•────•────•────            │  │
│     │                                             │  │
│     │    Line is too simple; misses the curve     │  │
│     └─────────────────────────────────────────────┘  │
│                                                      │
│     ┌─────────────────────────────────────────────┐  │
│     │           OVERFITTING                       │  │
│     │                                             │  │
│     │    •    •    •    •    •                   │  │
│     │       • • • • • •                           │  │
│     │    \ / \ / \ / \ / \ /                      │  │
│     │    \/  \/  \/  \/  \/                       │  │
│     │                                             │  │
│     │    Complex line follows every data point    │  │
│     │    (learns noise!)                          │  │
│     └─────────────────────────────────────────────┘  │
│                                                      │
│     ┌─────────────────────────────────────────────┐  │
│     │           GOOD FIT                          │  │
│     │                                             │  │
│     │    •    •    •    •    •                   │  │
│     │       • • • • • •                           │  │
│     │    ────•────•────•────•────•                │  │
│     │    \   \   \   \   \                        │  │
│     │                                             │  │
│     │    Balance: captures the curve without overfitting  │  │
│     └─────────────────────────────────────────────┘  │
│                                                      │
└──────────────────────────────────────────────────────┘
```

Comparison Table

Feature	Underfitting	Overfitting	Good Fit
Model Complexity	Too simple	Too complex	Just right
Training Performance	Poor	Excellent	Good
Test Performance	Poor	Poor	Good

Feature	Underfitting	Overfitting	Good Fit
Generalisation	Poor	Poor	Good
Cause	Too few features, too simple model	Too many features, too complex model	Balanced approach

---

8. Hyperparameter Tuning

What are Hyperparameters?

> **Hyperparameters are configurable settings of a model that are set before training. Unlike model parameters, they are not learned from the data.**

Examples of Hyperparameters

Algorithm	Hyperparameter	Description
K-NN	k	The number of neighbours
Decision Tree	Max depth	How deep the tree can grow
Decision Tree	Min samples per leaf	Minimum samples required in a leaf node
Linear Regression	(None typically)	

What is Hyperparameter Tuning?

> **Hyperparameter tuning is the process of finding the optimal hyperparameter values to improve model performance and avoid overfitting or underfitting.**

```text
┌─────────────────────────────────────────────────────────────────┐
│              HYPERPARAMETER TUNING PROCESS                     │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │  Step 1: Choose a hyperparameter to tune                │  │
│    │  (e.g., k in K-NN)                                     │  │
│    └─────────────────────────────────────────────────────────┘  │
│                          │                                      │
│                          ▼                                      │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │  Step 2: Try different values                          │  │
│    │  (k = 1, 3, 5, 7, 9, ...)                            │  │
│    └─────────────────────────────────────────────────────────┘  │
│                          │                                      │
│                          ▼                                      │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │  Step 3: Evaluate model performance                     │  │
│    │  (Use metrics: accuracy, precision, recall, F1)        │  │
│    └─────────────────────────────────────────────────────────┘  │
│                          │                                      │
│                          ▼                                      │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │  Step 4: Choose the best value                         │  │
│    │  (The one that gives the best performance)             │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Example: Tuning k in K-NN

k Value	Accuracy	Overfitting/Underfitting
k = 1	High on training, low on test	Overfitting (sensitive to noise)
k = 3	Good	Balanced
k = 5	Good	Balanced
k = 10	Lower	Slight underfitting
k = 20	Much lower	Underfitting (too general)

Choosing k: We would select k = 3 or 5 based on validation performance.

---

9. Activity: "Calculate the Metrics"

Instructions

1. You are given a **confusion matrix** for an email spam filter
2. Calculate:
   - **Accuracy**
   - **Precision** (for the Spam class)
   - **Recall** (for the Spam class)
3. Interpret the results

Confusion Matrix

```text
                        Predicted
                  Spam         Not Spam
            ┌──────────────────────────────────────────┐
            │              │              │              │
   Actual   │      TP      │      FN      │              │
   Spam     │      80      │      20      │              │
            ├──────────────┼──────────────┤              │
            │              │              │              │
   Actual   │      FP      │      TN      │              │
   Not Spam │      10      │     190      │              │
            └──────────────┴──────────────┘              │
```

Calculations

Metric	Formula	Calculation	Result
Accuracy	(TP + TN) / Total	(80 + 190) / 300	90.0%
Precision	TP / (TP + FP)	80 / (80 + 10)	88.9%
Recall	TP / (TP + FN)	80 / (80 + 20)	80.0%

Interpretation

- **Accuracy = 90%** : Overall, 90% of predictions are correct

- **Precision = 88.9%** : When the model predicts spam, it is correct 88.9% of the time
- **Recall = 80%** : The model catches 80% of actual spam emails

Question: Which metric is most important here? (Precision—we don't want to mark legitimate emails as spam!)

---

10. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Recap & Hook	10 min	Recap K-NN and Decision Trees; present the medical diagnosis scenario to highlight limitations of accuracy
Lecture: Evaluation Metrics	20 min	Confusion matrix; TP, TN, FP, FN; accuracy, precision, recall
Activity	15 min	"Calculate the Metrics" — students calculate metrics from a confusion matrix
Lecture: Overfitting, Underfitting & Tuning	10 min	Overfitting vs. underfitting; F1 score; hyperparameter tuning
Consolidation	5 min	Recap; preview next lesson

Discussion Questions

Question	Purpose
"Why can accuracy be misleading?"	Understand limitations
"What is the difference between precision and recall?"	Differentiate metrics
"When would you prioritise recall over precision?"	Apply concepts
"What is the F1 score and why is it useful?"	Understand balance
"What is overfitting and how can we prevent it?"	Address training issues

---

11. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Activity	Evaluate ability to calculate metrics from confusion matrix
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

1. What is the difference between precision and recall?
2. Why is accuracy misleading with imbalanced data?

3. What is overfitting and what causes it?
4. What is a hyperparameter? Give one example.

---

12. Differentiation

Level	Strategy
Support	Use strong, memorable examples; focus on "in words" definitions; step through activity carefully
Challenge	Research precision-recall curves; explore cross-validation for hyperparameter tuning

---

13. Resources

- **Presentation** (Slide Deck)
- **eBook** (A4.3.3 with examples)
- **Worksheet:** Confusion matrix and metrics calculations
- **Worksheet Answers**
- **Quizlet** (A4.3.3 vocabulary flashcards)

---

14. Summary: Key Takeaways

Concept	Key Point
Accuracy	Proportion of correct predictions; can be misleading
Precision	When model says "yes", how often is it right?
Recall	How good is model at finding all "yes" cases?
F1 Score	Harmonic mean of precision and recall (balance)
Confusion Matrix	TP, TN, FP, FN — summarises performance
Overfitting	Too complex; learns noise
Underfitting	Too simple; fails to learn
Hyperparameter	Settings chosen before training
Hyperparameter Tuning	Finding optimal hyperparameter values

```text
┌─────────────────────────────────────────────────────────────┐
│                    KEY REMINDERS                            │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│    ✅ Accuracy can be misleading (imbalanced data)          │
│    ✅ Precision = correctness of positive predictions       │
│    ✅ Recall = completeness of positive predictions         │
│    ✅ F1 = balance between precision and recall             │
│    ✅ Overfitting = learns noise                            │
│    ✅ Underfitting = too simple                             │
│    ✅ Hyperparameter = setting chosen before training       │
```

```
│     ✅ Tuning = finding best hyperparameters                 │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

> *"Evaluating a model is a nuanced process—accuracy alone is not enough. Precision, recall, F1, and understanding overfitting are all essential for building reliable machine learning systems."*